JAVA FULL STACK DEVELOPMENT INTERNSHIP

EXPENSE TRACKER

Project Report

Submitted by	Piyush
Project	Personal Expense Tracker
Technology	Java 17, Spring Boot, React, MySQL
Submission date	20 October 2026
Institution / organization	_____

A full-stack web application for account-based expense tracking

1. Project overview

Expense Tracker is a personal finance web application. A user can create an account, sign in, record expenses, review the expense list and remove records. The backend stores accounts and expenses in MySQL and scopes expense operations to the authenticated account. The React interface provides a responsive form and spending summary.

Objectives

- Build a Java and React application with a relational database.
- Secure account passwords and protect expense endpoints with session authentication.
- Provide REST endpoints for registration, sign-in, and expense management.
- Validate user input and return useful errors.
- Restrict administrator routes using role-based authorization.

2. Technology stack

Layer	Tools	Purpose
Frontend	React, Vite, JavaScript, CSS	Responsive user interface and API calls
Backend	Java 17, Spring Boot, Spring Security, Spring Data JPA, Maven	REST API, authentication, validation, and persistence
Database	MySQL	Stores user accounts and expenses

3. Application architecture

The project separates responsibilities into backend packages. The auth package handles registration, login, user accounts, and administration; expense contains the expense entity, repository, service, and controller; config contains Spring Security and CORS settings; common contains API error handling. The frontend communicates with the backend through a shared API module.

Request flow

Browser UI -> React API module -> Spring REST controller -> service and repository -> MySQL

4. Implemented features

Authentication and authorization

Users register with a name, email, and password. Passwords are stored as BCrypt hashes. Login establishes a server-side session; the frontend sends session cookies with API requests. The current-user endpoint restores the signed-in account after a page reload, and logout clears the session. Protected expense routes require authentication. The admin summary route is limited to the ADMIN role.

Expense management

Each expense has a description, amount, category, date, and owning account. Signed-in users can list their expenses, add a record, and delete their own record. The dashboard shows the recorded total and recent expense list. Data remains in MySQL after a browser refresh or a later sign-in.

Validation and error handling

Request DTOs and expense fields use validation constraints such as required values, email format, and a positive amount. A shared API exception handler formats validation and application errors for the client. The frontend turns common login and registration failures into readable messages.

5. REST API overview

Method	Endpoint	Purpose / access
POST	/api/auth/register	Create a user account; public
POST	/api/auth/login	Sign in and establish session; public
GET	/api/auth/me	Return current signed-in user
POST	/api/auth/logout	End the current session
GET	/api/expenses	List current user expenses; authenticated
POST	/api/expenses	Create expense; authenticated
DELETE	/api/expenses/{id}	Delete owned expense; authenticated
GET	/api/admin/summary	Admin summary; ADMIN role

Session-based requests use browser cookies with credentials included. JSON request and response details are provided in the separate API documentation.

6. Database design

The database contains two main tables. user_accounts stores identity and role details; expenses stores the user's expense records. Each expense references its owner using user_id. The SQL schema is included as docs/database-schema.sql.

Table	Representative columns	Notes
user_accounts	id, name, email, password, role	Email identifies the account; password contains a hash; role supports USER or ADMIN access.
expenses	id, description, amount, category, expense_date, user_id	user_id links the record to its owner; amount uses decimal precision for currency.

7. Running the project locally

Prerequisites: Java 17, Maven, Node.js with npm, and a running MySQL server. Create the database and tables using docs/database-schema.sql or allow the configured JPA schema update for local development. Set the database URL, username, and password in backend/src/main/resources/application.yml.

Start the backend from the backend folder with `mvn spring-boot:run`. Start the frontend from the frontend folder with `npm install` followed by `npm run dev`. Open the Vite local URL printed in the terminal. Keep both server terminals running while using the app.

8. Verification and current delivery status

During local development, the backend compiled successfully, the frontend development server started, and the application was used to register/sign in, add and delete expenses, preserve records in the database across refresh/sign-in, and check the administrator summary. These are development observations, not a claim of a formal automated test suite.

The application is currently a local project. A public GitHub repository, live deployment, LinkedIn post, and recorded 3–5 minute demo must be added after publishing/recording; they are not represented as completed here.

Conclusion

The project demonstrates the core full-stack workflow for a personal expense tracker: a responsive client, Spring Boot REST services, session-based authentication, role checks, validation, and MySQL persistence. The structure supports further development, including editing expenses and richer reports.

Submission checklist

- Source code and README
- Database schema and API documentation
- This project report
- Record a 3–5 minute demo video
- Publish to a public GitHub repository and deploy the frontend/backend
- Add the GitHub, live, and LinkedIn links to the final submission